

Práctica 1: Clasificador k NN y Selección de Atributos

Introducción y objetivos

En esta primera práctica se va a trabajar con un clasificador sencillo, para recordar los procedimientos para entrenar, validar y probar un modelo. Además se trabajará con técnicas de selección de características y se analizará su efecto.

Los objetivos de esta entrega son los siguientes:

- Implementar un algoritmo de clasificación k Nearest Neighbors (k NN) desde cero.
- Evaluar y optimizar los hiperparámetros del modelo.
- Aplicar técnicas de selección de atributos (Variance Threshold, F -score y mRMR) para intentar mejorar dicha predicción.

Conjunto de datos y preprocesado

El conjunto de datos que se va a utilizar para esta primera práctica es `breast_cancer`, un conjunto de clasificación clásico en el ámbito del aprendizaje automático. Se deberán llevar a cabo las siguientes acciones para poder trabajar con este conjunto:

1. Cargar el conjunto de datos desde `sklearn.datasets`.
2. El dataset se partirá de forma que quede un 70 % para entrenar y un 30 % para pruebas.
3. Normalizar los datos.

***Nota.** ¿Es importante el orden en el que se divida el conjunto y se normalicen los datos? ¿Por qué?*

Implementación de la clase `KNNClassifier` y validación

Completar el módulo `KNNClassifier.py` para realizar una **implementación** propia de k NN, definiendo los siguientes métodos:

1. Constructor: `__init__(self, k=3, distance_metric='euclidean')`
2. Entrenamiento: `fit(self, X, y)`, ajuste y almacenamiento del modelo.
3. Predicción: `predict(self, X)`, obtención de la nueva etiqueta en base a la etiqueta mayoritaria de los vecinos.

Se puede verificar el código implementado ejecutando los tests proporcionados. Para ello, basta con ejecutarlos desde terminal usando `pytest`. Se debe tener en cuenta que estos tests no son exhaustivos.

Tras implementar la clase, para **validar** la codificación se utilizará el conjunto `breast_cancer` ya preprocesado, de forma que:

1. Se predecirán las etiquetas de los datos de test utilizando un número de vecinos k fijo pero arbitrario.

2. Se compararán las predicciones del modelo propio con las obtenidas mediante el método de `scikit-learn` (`KNeighborsClassifier`) usando los mismos hiperparámetros y calculando el error medio entre ambas salidas.

***Nota.** Si el problema al que nos enfrentamos es un problema desbalanceado, ¿qué pasaría con la predicción de k NN? ¿Habría que hacer algún preproceso previo o modificación del algoritmo?*

Optimización de Hiperparámetros

Para seleccionar el mejor modelo, es muy importante ajustar bien los valores de los hiperparámetros del método a utilizar.

1. **Selección de k Óptimo:** Determinar k_{opt} mediante validación cruzada sobre el conjunto de entrenamiento.
2. **Curva de Aprendizaje:** Implementar mediante `matplotlib` la representación visual de la precisión (*accuracy*) frente al número de vecinos k tanto para el conjunto de entrenamiento como para el de test.
3. **Evaluación de otras métricas:** Se debe probar a cambiar la distancia por la distancia de Minkowski, definida como:

$$D(x, y) = \left(\sum_{i=1}^d |x_i - y_i|^p \right)^{1/p}$$

Para evaluar su efecto, se puede representar gráficamente la precisión (*accuracy*) en función del parámetro p ($p \in \{1; 2; \dots, 10\}$), tanto para el conjunto de entrenamiento como de test, analizando la influencia del mismo.

***Nota.** La distancia, y en este caso el parámetro p , son también hiperparámetros del modelo. ¿Sería necesario incluirlos en el proceso de validación cruzada?*

Selección de Atributos

Por último, se va a reducir la dimensión del conjunto de datos mediante métodos de selección de características, para lo cual se van a probar distintas opciones.

1. **Filtros Univariados y de Varianza:**
 - Aplicar `VarianceThreshold` con distintos umbrales $u \in [0, 1]$.
 - Utilizar `SelectKBest` junto con el test F (`f_classif`) para seleccionar los K mejores atributos.

***Nota.** Analizar cómo afecta la selección de atributos al *accuracy* en test del modelo previamente seleccionado. El resultado puede ser una gráfica del *accuracy* frente al umbral u o al número de atributos K , según el caso.*

¿Tiene sentido en `VarianceThreshold` los valores $u = 0$ y $u = 1$?

2. **Método mRMR (Minimum Redundancy Maximum Relevance):** Implementar en `mRMR.py` la selección iterativa de atributos maximizando la relevancia con la variable objetivo y y minimizando la redundancia entre atributos mediante Información Mutua $I(\cdot)$:

$$\max_{X_i \notin S} \left[I(X_i; y) - \frac{1}{|S|} \sum_{X_j \in S} I(X_i; X_j) \right]$$

Para este método también se proporcionan algunos tests básicos para comprobar su correcta implementación.

3. **Evaluación Comparativa:** Pintar en una misma gráfica el *accuracy* del modelo k NN en el conjunto de test como función del número de atributos seleccionados K para los distintos métodos.

Nota. ¿Cuál de estos métodos se podría considerar el mejor para este problema?

Criterios de corrección

Se debe entregar un archivo comprimido (*.zip*) que contenga los archivos `KNNClassifier.py` y `mRMR.py` con las implementaciones solicitadas, así como un jupyter notebook (*.ipynb*) con las pruebas y análisis planteados. Esta entrega se calificará como APTO/NO APTO.

La calificación de esta primera práctica, en caso de obtener un APTO en la entrega, será la obtenida en la prueba individual realizada tras la entrega.